

gubot_one — Vollständige Projektanalyse

Analyse vom 18.03.2026

Pakete geprüft: `src/face_tracker` · `src/gubot_gazebo` · `src/gubot_one` · `src/gubot_one_bringup` · `src/gubot_one_description` · `src/nerf_launch_system`

Executive Summary

 **Gesamtzustand: Entwicklungsfähig — aber mit kritischen Fehlern**

Die Projektstruktur hat sich während des Umbaus in einen inkonsistenten Zustand entwickelt. Es gibt **4 kritische Bugs** die einen Start verhindern würden, **6 Duplikat-Gruppen** und mehrere fehlende Sensor-Integrationen.

Kategorie	Status
Gazebo Harmonic (gz-sim) Setup	 Vorhanden, 3 Bugs
Gazebo Classic Setup	 Gebrochen — falscher RSP-Pfad
Real-Hardware Launch	 Funktioniert
IMU (Simulation)	 Bridge auskommentiert
Kamera (Simulation)	 Läuft, aber veraltetes Format
Nerf-Launcher URDF	 Flywheel-Joints als <code>fixed</code> deklariert
Face Tracker	 Vollständig implementiert
Duplikate	 6 Duplikat-Gruppen gefunden

Paketstruktur-Übersicht

```
my_new_robot/src/
├─ gubot_one/           ⚠️ Meta-Paket – enthält BROKEN Duplikate
├─ gubot_one_bringup/   ✅ Haupt-Bringup (Hardware + Sim)
├─ gubot_one_description/ ✅ URDF/Xacro (primäre Quelle)
├─ gubot_gazebo/        ✅ Gazebo Harmonic Simulation (neuerer Ansatz)
├─ face_tracker/        ✅ Gesichtserkennung + Steuerung
└─ nerf_launch_system/  🟡 Hardware-Interface + URDF (Duplikat-Problem)
```

Paket-Rollen (Soll vs. Ist)

Paket	Soll-Rolle	Ist-Zustand
gubot_one	Meta-Umbrella Paket	⚠️ Enthält gebrochene Launch/Description-Kopien
gubot_one_bringup	Hardware + Sim Bringup	✅ Haupt-Bringup, korrekt
gubot_one_description	URDF-Quelle	✅ Primäre URDF-Quelle
gubot_gazebo	Gazebo Sim (separiert)	✅ Neuerer, saubererer Ansatz
face_tracker	Kamera + KI-Steuerung	✅ Vollständig
nerf_launch_system	Launcher HW + URDF	🟡 Duplikat-URDFs

Gazebo Harmonic (gz-sim / Ignition) Setup

Launch-Ketten

Kette A — gubot_one_bringup (Haupt-Simulation):

```
launch_sim.launch.py
├─ rsp.launch.py → robot.urdf.xacro
│   │   └─ robot_core.xacro      (Chassis, Räder, IMU-Sensor)
│   │   └─ ros2_control.xacro    (GazeboSimSystem Plugin)
│   │   └─ camera.xacro          (Kamera-Sensor)
│   │   └─ face.xacro            (Face-Link)
│   └─ nerf_launcher.xacro
│       └─ launcher.urdf.xacro (Nerf URDF)
├─ gz_sim.launch.py (ros_gz_sim)
├─ ros_gz_bridge      (Clock, Scan, Camera)
├─ Controller Spawner (diff_cont, joint_broad, imu_broadcaster, tilt, shooter, arming)
└─ imu_filter_madgwick
```

Kette B — gubot_gazebo (neuerer, saubererer Ansatz):

```
simulation.launch.py
├─ gz_sim.launch.py (mit ogre render-engine)
├─ gz_bridge (YAML-Konfiguration)
├─ spawn_robot.launch.py
│   └─ rsp.launch.py (aus gubot_one_bringup)
```

- controller Spawner
- imu_filter (aus imu_filter.yaml)

ros2_control Plugin (Harmonic)

```
<!-- In ros2_control.xacro - SIM MODE -->
<plugin>gz_ros2_control/GazeboSimSystem</plugin>

<!-- Gazebo Plugin (PROBLEM: NICHT KONDITIONELL!) -->
<gazebo>
  <plugin filename="gz_ros2_control-system"
    name="gz_ros2_control::GazeboSimROS2ControlPlugin">
```

● BUG #1 — gz_ros2_control Plugin nicht konditionell

Der <gazebo>-Block am Ende von ros2_control.xacro ist **nicht** in ein <xacro:if value="\$(arg sim_mode)"> eingeschlossen. Dadurch wird der Ignition-Plugin-Block auch für **echte Hardware** in die URDF eingetragen
→ Fehler beim Hardware-Start.

Fix: Gesamten <gazebo>-Block in

<xacro:if value="\$(arg sim_mode)"> ... </xacro:if> einschließen.

● BUG #2 — IMU-Bridge auskommentiert

In **beiden** Harmonic-Launch-Dateien (launch_sim.launch.py und spawn_robot.launch.py) ist die IMU-Bridge auskommentiert:

```
# "/imu_sensor/imu_data@sensor_msgs/msg/Imu[gz.msgs.IMU"
```

Das bedeutet: Die IMU-Daten des Simulators gelangen **nicht zu ROS** → imu_filter_madgwick empfängt keine Daten → Navigation funktioniert nicht in der Simulation.

Fix: Bridge-Zeile einkommentieren (das # entfernen) in beiden Dateien.

Gazebo Classic Setup

Launch-Ketten

Kette A (KAPUTT) — `gubot_one_bringup`:

```
gz_classic_launch_sim.launch.py
  → rsp.launch.py ←  FALSCHER RSP!
    → robot.urdf.xacro
      └─ ros2_control.xacro ← lädt gz_ros2_control (HARMONIC!)
```

Kette B (KORREKT) — `via rsp_classic`:

```
[manuell oder via eigenem launch]
  → rsp_classic.launch.py
    → robot_classic.urdf.xacro
      └─ robot_core_classic.xacro (IMU mit libgazebo_ros_imu_sensor.so )
      └─ gz_classic_ros2_control_sim.xacro (gazebo_ros2_control/GazeboSystem )
      └─ nerf_launcher_classic.xacro
        └─ launcher_gazebo_classic.urdf.xacro
      └─ camera.xacro
      └─ face.xacro
```

BUG #3 — `gz_classic_launch_sim.launch.py` verwendet falschen RSP

`gz_classic_launch_sim.launch.py` ruft `rsp.launch.py` auf (mit Argument `use_gazebo_classic:=true`). Aber `rsp.launch.py` lädt **immer** `robot.urdf.xacro`, welches `ros2_control.xacro` einbindet — und das enthält das **Ignition/Harmonic-Plugin** (`gz_ros2_control-system`), nicht das Classic-Plugin (`libgazebo_ros2_control.so`).

Fix: In `gz_classic_launch_sim.launch.py` ersetzen:

`rsp.launch.py` → `rsp_classic.launch.py`

und die Übergabe von `use_gazebo_classic` entfernen (nicht nötig für `rsp_classic`).

ros2_control Plugins im Vergleich

Datei	Plugin	Zweck
<code>ros2_control.xacro (sim_mode=true)</code>	<code>gz_ros2_control/GazeboSimSystem</code>	Harmonic Sim
<code>gz_classic_ros2_control_sim.xacro</code>	<code>gazebo_ros2_control/GazeboSystem</code>	Classic Sim
<code>gz_classic_ros2_control.xacro</code>	Beide (konditionell)	Real + Classic (veraltet)
<code>ros2_control.xacro (sim_mode=false)</code>	<code>diffdrive_arduino/DiffDriveArduinoHardware</code>	Echte Hardware

Sensor-Analyse

IMU — ICM-20948

Modus	Implementierung	Status
-------	-----------------	--------

Real Hardware	State-Interfaces in <code>ros2_control1.xacro</code> (RealRobot) → <code>imu_broadcaster</code> → <code>imu_filter_madgwick</code>	✅ Korrekt
Harmonic Sim	<code>robot_core.xacro</code> : <code><sensor type="imu"> + <topic>imu_sensor/imu_data</topic></code> — aber Bridge auskommentiert	❌ Bridge fehlt
Classic Sim	<code>robot_core_classic.xacro</code> : <code>libgazebo_ros_imu_sensor.so</code> Plugin + <code><remapping>~/out:=imu/data</remapping></code>	⚠️ Remapping prüfen

Harmonic IMU Topic-Kette (nach Fix):

```
Gazebo Sensor → /imu_sensor/imu_data (gz)
→ ros_gz_bridge → /imu_sensor/imu_data (ROS)
→ imu_broadcaster → /imu_broadcaster/imu
→ imu_filter_madgwick → /imu/data
```

Classic IMU Topic-Kette:

```
libgazebo_ros_imu_sensor → /imu/data_raw (oder /imu_link/imu)
→ imu_filter_madgwick → /imu/data
```

⚠️ Remapping in `launch_robot.launch.py` mapped `/imu/data_raw` → `/imu_broadcaster/imu`. Für Classic muss das Topic-Mapping der Classic-Plugins stimmen.

Kamera

Datei	Format	Status
<code>camera.xacro</code>	Gazebo Classic Sensor-XML in <code><gazebo reference></code>	🟡 Kompatibel, aber veraltet

Was fehlt für Harmonic: Die aktuelle `camera.xacro` verwendet das alte Gazebo Classic Format. Für Gazebo Harmonic sollte die Kamera idealerweise mit dem `gz::sim::systems::Camera` SDF-Plugin direkt definiert werden. Die aktuelle Implementierung funktioniert via Kompatibilitäts-Layer, ist aber nicht die saubere Methode.

Camera Bridge (korrekt eingerichtet):

```
"/camera/image_raw@sensor_msgs/msg/Image[gz.msgs.Image",
"/camera/camera_info@sensor_msgs/msg/CameraInfo[gz.msgs.CameraInfo",
```

Depth Camera (`depth_camera.xacro`): Existiert, ist aber in `robot.urdf.xacro` auskommentiert.

```
<!-- include filename="depth_camera.xacro" -->
```

Nerf Launcher Joints

❌ **BUG #4 — Flywheel-Joints als `fixed` deklariert**

In `launcher.urdf.xacro` (Harmonic-Version) sind die Flywheel-Joints als `type="fixed"` definiert:

```
<joint name="flywheel_right_joint" type="fixed">
<joint name="flywheel_left_joint" type="fixed">
```

Aber in `ros2_control1.xacro` (IgnitionSystem) werden sie mit Velocity-Interfaces registriert:

```
<joint name="flywheel_left_joint">
  <command_interface name="velocity"/>
</joint>
```

Fixed Joints können nicht angesteuert werden → Controller-Spawner für Flywheel wird scheitern oder einen Fehler ausgeben. In `launcher_gazebo_classic.urdf.xacro` haben die Joints ebenfalls `type="fixed"`.

Fix: Joint-Typ in `launcher.urdf.xacro` auf `type="continuous"` ändern:

```
<joint name="flywheel_left_joint" type="continuous">
  <axis xyz="0 1 0"/>
  <limit effort="10" velocity="100.0"/>
</joint>
```

Duplikat-Analyse

Duplikat-Gruppe 1 — Launch-Dateien in `gubot_one` (KRITISCH)

Betroffen:

`src/gubot_one/launch/launch_sim.launch.py` (288 Zeilen)

`src/gubot_one_bringup/launch/launch_sim.launch.py` (288 Zeilen)

Diese Dateien sind **nahezu identisch**. Einzige Unterschiede: `package_name = "gubot_one"` VS `"gubot_one_bringup"` und einige MESA-Env-Variablen. Die `gubot_one`-Version referenziert jedoch Configs und Worlds die in diesem Paket **nicht existieren** → Laufzeit-Fehler.

Gleiches gilt für `gz_classic_launch_sim.launch.py` in beiden Paketen.

Lösung: Launch-Dateien aus `gubot_one` komplett entfernen. `gubot_one` soll ein reines Meta-Paket sein.

Duplikat-Gruppe 2 — Description-Dateien in `gubot_one`

Betroffen:

`src/gubot_one/description/robot_core.xacro`

`src/gubot_one_description/description/robot_core.xacro`

Die `gubot_one`-Version ist sogar die **verbesserte Variante** — sie hat den korrekten `use_gazebo_classic`-Conditional für das IMU-Plugin. Diese Verbesserungen sollten in die primäre Datei in `gubot_one_description` gemergt werden.

Gleiches gilt für `gz_classic_ros2_control.xacro` in beiden Paketen — die `gubot_one`-Version referenziert jedoch `$(find gubot_one)/config/` was nicht existiert.

Lösung: Verbesserungen aus `gubot_one/description/` nach `gubot_one_description/description/` übertragen, dann Dateien aus `gubot_one` entfernen.

Duplikat-Gruppe 3 — Nerf Launcher URDF (zwei Varianten)

Datei	Unterschied
<code>launcher.urdf.xacro</code>	Mesh-Pfad: <code>package:///</code> · Masse: 0.1 kg · Inertia: 0.001
<code>launcher_gazebo_classic.urdf.xacro</code>	Mesh-Pfad: <code>file://\$(find ...)</code> · Masse: 0.001 kg · Inertia: 1e-06

Die klassische Version hat deutlich kleinere Inertia-Werte (für Stabilität in Classic) und nutzt `file://` Mesh-Pfade. Die Struktur ist ansonsten identisch. Diese können mit Xacro-Argumenten vereint werden (`use_gazebo_classic` arg für Pfad + Inertia), sind aber funktional als zwei getrennte Dateien vertretbar.

Duplikat-Gruppe 4 — Worlds-Dateien

Datei	Vorhanden in
-------	--------------

obstacles.world	gubot_gazebo · gubot_one_bringup · nerf_launch_system
empty.world	gubot_gazebo · nerf_launch_system
obstacles_classic.world	gubot_one_bringup

World-Dateien sollten in **einem** Paket (`gubot_gazebo`) zentralisiert werden.

Duplikat-Gruppe 5 — gazebo_params.yaml

`gubot_one_bringup/config/gazebo_params.yaml` und `nerf_launch_system/config/gazebo_params.yaml`
— wahrscheinlich identisch oder sehr ähnlich. Sollte in `gubot_gazebo/config/` konsolidiert werden.

Duplikat-Gruppe 6 — gz_classic_ros2_control.xacro (drei Versionen!)

Datei	Beschreibung	Problem
<code>gubot_one_description/description/gz_classic_ros2_control.xacro</code>	Full version mit sim+hw conditional	Referenziert <code>gubot_one_bringup/conf</code> ✓
<code>gubot_one/description/gz_classic_ros2_control.xacro</code>	Ähnlich, mit kommentierten Flywheel-Joints	Referenziert <code>gubot_one/config/</code> ✗ nicht vorhanden
<code>gubot_one_description/description/gz_classic_ros2_control_sim.xacro</code>	Nur-Sim Version (sauber, keine Conditionals)	✓ Korrekt, wird von <code>robot_classic.urdf.xacro</code> genutzt

✓ Was funktioniert gut

Face Tracker System (vollständig)

```
face_tracker/
├─ detect_face.py      – Gesichtserkennung via OpenCV/dlib
├─ follow_face.py      – Publish cmd_vel für Face-Following
├─ fire_at_face.py     – Steuert Nerf-Launcher bei Gesichtserkennung
├─ process_image.py    – Bildvorverarbeitung
├─ fake_face_publisher.py – Sim-Testing ohne Kamera
├─ register_face.py    – Gesichtsdaten registrieren
├─ udp_cam_sender.py   – UDP Kamera-Streaming (Windows → WSL)
└─ udp_cam_receiver.py – UDP Empfänger

gubot_one_bringup/scripts/
├─ nerf_joy.py         – Manueller Joystick → Nerf-Steuerung
└─ nerf_teleop.py     – Keyboard Teleop für Nerf
```

Real-Hardware Launch (vollständig)

```
launch_robot.launch.py → diffdrive_arduino (Pico)
                        → NerfSystem (Arduino Leonardo)
                        → imu_broadcaster → imu_filter_madgwick
                        → twist_mux (Joystick > Face > Keyboard)
                        → Sequenzielles Controller-Spawning
                        → OpaqueFunction: fake_hardware=true → kein 3s-Delay
```

ros2_control Joint-Mapping (Hardware)

Joint	Controller	Hardware
left_wheel_joint	diff_cont	Pico (DiffDriveArduino)
right_wheel_joint	diff_cont	Pico (DiffDriveArduino)
imu_sensor	imu_broadcaster	Pico (ICM-20948)
tilt_joint	tilt_controller	Arduino Leonardo
shooter_joint	shooter_controller	Arduino Leonardo
system_arming_joint	arming_controller	Arduino Leonardo

gubot_gazebo Paket (neuerer Ansatz — empfohlen)

- Sauberere Bridge-Konfiguration via YAML-Datei
- `imu_filter` Parameter aus YAML (nicht inline)
- WSLg-spezifische Optimierungen (D3D12, NVIDIA-Adapter)
- Korrekte `IGN_GAZEBO_RESOURCE_PATH` für alle nötigen Pakete

🔧 Priorisierte Fix-Liste

● Kritisch (müssen vor einem Start behoben werden)

#	Problem	Datei	Fix
1	IMU-Bridge auskommentiert	launch_sim.launch.py, spawn_robot.launch.py	# "/imu_sensor/imu_data..." einkommentieren
2	gz_ros2_control Plugin nicht konditionell	ros2_control.xacro	<gazebo>-Block in <xacro:if sim_mode> einschließen
3	Classic-Launch nutzt falschen RSP	gz_classic_launch_sim.launch.py	rsp.launch.py → rsp_classic.launch.py ersetzen
4	Flywheel-Joints als fixed	launcher.urdf.xacro	Typ auf continuous ändern, Axis und Limit hinzufügen

● Wichtig (für Stabilität und Wartbarkeit)

#	Problem	Fix
5	gubot_one/launch/ enthält broken Duplikate	Alle Launch/Description Dateien aus gubot_one entfernen
6	robot_core.xacro in gubot_one_description fehlt use_gazebo_classic IMU-Conditional	Feature aus gubot_one/description/robot_core.xacro übernehmen
7	gz_classic_ros2_control.xacro in gubot_one referenziert nicht-existierende Config	Datei entfernen (nach Merge von Verbesserungen)
8	World-Dateien in 3 Paketen verstreut	In gubot_gazebo/worlds/ konsolidieren

● Nice-to-have (Verbesserungen)

#	Problem	Fix
9	camera.xacro nutzt veraltetes Sensor- Format	Auf Gazebo Harmonic SDF-Format updaten
10	depth_camera.xacro auskommentiert	Entscheidung: aktivieren oder entfernen
11	launcher.urdf.xacro und Classic- Version	Mit xacro:arg use_gazebo_classic zusammenführen
12	IMU-Remapping für Classic prüfen	Sicherstellen dass libgazebo_ros_imu_sensor auf /imu/data_raw published

Fix #1 — IMU Bridge einkommentieren

Datei: src/gubot_one_bringup/launch/launch_sim.launch.py

Und: src/gubot_gazebo/launch/spawn_robot.launch.py

```
# VORHER:
bridge = Node(
    package="ros_gz_bridge",
    executable="parameter_bridge",
    arguments=[
        "/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock",
        "/scan@sensor_msgs/msg/LaserScan[gz.msgs.LaserScan",
        "/camera/image_raw@sensor_msgs/msg/Image[gz.msgs.Image",
        "/camera/camera_info@sensor_msgs/msg/CameraInfo[gz.msgs.CameraInfo",
        # "/imu_sensor/imu_data@sensor_msgs/msg/Imu[gz.msgs.IMU", ← PROBLEM
    ],
)

# NACHHER:
bridge = Node(
    package="ros_gz_bridge",
    executable="parameter_bridge",
    arguments=[
        "/clock@rosgraph_msgs/msg/Clock[gz.msgs.Clock",
        "/scan@sensor_msgs/msg/LaserScan[gz.msgs.LaserScan",
        "/camera/image_raw@sensor_msgs/msg/Image[gz.msgs.Image",
        "/camera/camera_info@sensor_msgs/msg/CameraInfo[gz.msgs.CameraInfo",
        "/imu_sensor/imu_data@sensor_msgs/msg/Imu[gz.msgs.IMU", ← FIX
    ],
)
```

Fix #2 — gz_ros2_control Plugin konditionell machen

Datei: src/gubot_one_description/description/ros2_control.xacro

```
<!-- VORHER (am Ende der Datei): -->
<gazebo>
  <plugin filename="gz_ros2_control-system"
    name="gz_ros2_control::GazeboSimROS2ControlPlugin">
    <parameters>$(find gubot_one_bringup)/config/my_controllers.yaml</parameters>
    <parameters>$(find gubot_one_bringup)/config/gaz_ros2_ctl_use_sim.yaml</parameters>
  </plugin>
</gazebo>

<!-- NACHHER: -->
<xacro:if value="$(arg sim_mode)">
  <gazebo>
    <plugin filename="gz_ros2_control-system"
      name="gz_ros2_control::GazeboSimROS2ControlPlugin">
      <parameters>$(find gubot_one_bringup)/config/my_controllers.yaml</parameters>
```

```

        <parameters>${(find
gubot_one_bringup)/config/gaz_ros2_ctl_use_sim.yaml}</parameters>
    </plugin>
</gazebo>
</xacro:if>

```

Fix #3 — Classic Launch: richtiger RSP

Datei: src/gubot_one_bringup/launch/gz_classic_launch_sim.launch.py

```

# VORHER:
rsp = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(
        [os.path.join(get_package_share_directory(package_name),
            "launch", "rsp.launch.py")] # ← FALSCH
    ),
    launch_arguments={
        "use_sim_time": use_sim_time,
        "use_gazebo_classic": "true", # ← ignoriert von rsp.launch.py!
    }.items(),
)

# NACHHER:
rsp = IncludeLaunchDescription(
    PythonLaunchDescriptionSource(
        [os.path.join(get_package_share_directory(package_name),
            "launch", "rsp_classic.launch.py")] # ← KORREKT
    ),
    launch_arguments={
        "use_sim_time": use_sim_time,
    }.items(),
)

```

Fix #4 — Flywheel-Joints als continuous

Datei: src/nerf_launch_system/description/urdf/launcher.urdf.xacro

```

<!-- VORHER: -->
<joint name="flywheel_right_joint" type="fixed">
    <parent link="motor_right" />
    <child link="flywheel_right" />
    ...
</joint>

<!-- NACHHER: -->
<joint name="flywheel_right_joint" type="continuous">
    <parent link="motor_right" />
    <child link="flywheel_right" />
    <origin xyz="0.0 -7.8e-06 -0.0175" rpy="1.5707963 0 0" />
    <axis xyz="0 0 1"/>

```

```

<limit effort="10.0" velocity="100.0"/>
<dynamics damping="0.01" friction="0.01"/>
</joint>

```

```

<!-- Gleiches für flywheel_left_joint -->

```

Abschließende Bewertung

Bereich	Bewertung	Priorität
Gazebo Harmonic Launch-Infrastruktur	● 80% fertig	IMU-Bridge + Plugin-Conditional
Gazebo Classic Launch-Infrastruktur	● 60% fertig	RSP-Fehler beheben
Real Hardware	● 95% fertig	Produktiv einsetzbar
Nerf URDF (Simulation)	● 70% fertig	Flywheel-Joints fixen
Face Tracker	● 90% fertig	Vollständig implementiert
Paket-Sauberkeit	● 50%	Duplikate aus gubot_one entfernen
Sensor-Integration Real	● 90%	IMU, Kamera, Nerf vorhanden
Sensor-Integration Sim	● 70%	IMU-Bridge fehlt

✓ Empfohlene Reihenfolge der Fixes:

1. Fix #1 (IMU Bridge) — 2 Minuten, sofort testen
2. Fix #3 (Classic RSP) — 5 Minuten
3. Fix #2 (Plugin konditionell) — 10 Minuten
4. Fix #4 (Flywheel Joints) — 15 Minuten
5. Duplikate aus gubot_one entfernen — 30 Minuten
6. robot_core.xacro IMU-Conditional mergen — 15 Minuten

Nach diesen 6 Schritten ist das Projekt in einem sauberen, funktionsfähigen Zustand.